# Programming Languages II

Zoe Paraskevopoulou

School of Electrical and Computer Engineering
National Technical University of Athens

2025



# Course Subject

## Programming Language Theory

- **Static semantics**
  Properties of a programming language that can be checked and enforced before the program executes
- **Dynamic semantics**
  What a program does when it executes
- Program **correctness**
- Program **transformations**
- Static **analysis** of programs

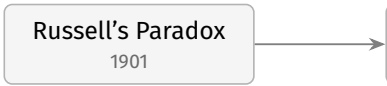



xkcd.com/568

# A Brief History



# A Brief History



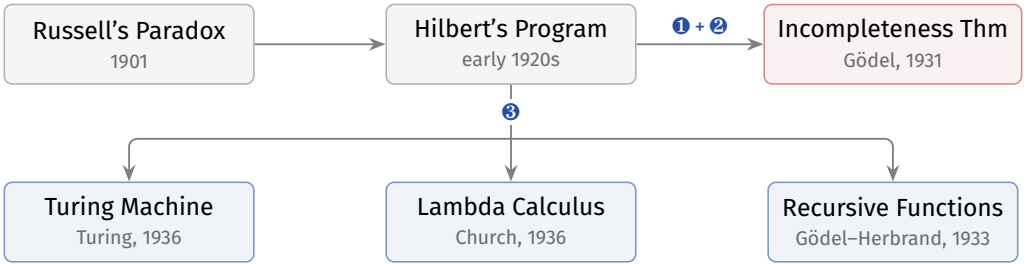

**Foundations of mathematics:** statements must be written in a **formal language** and proved by well-defined rules.

**❶ Consistency**
Only true propositions can be proved

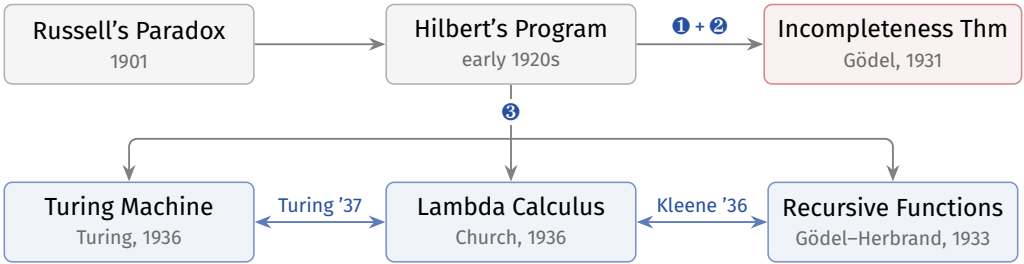
**❷ Completeness**
All true propositions can be proved

**❸ Decidability**
Truth can be decided algorithmically

# A Brief History



# A Brief History




Halting problem

No method to decide $\lambda$-expression equivalence

**All** non-trivial semantic properties are undecidable
Rice's thm, 1951

# $\lambda$-Calculus (Lambda Calculus)

- Created by Church in the 1930s
- A formal system in mathematical logic for defining computability
- A universal model of computation (equivalent to Turing machines)
- The foundation of functional languages (and beyond)



variable binding

functions

parameter passing



scoping

expressions

type systems

**Foundations of Mathematics**

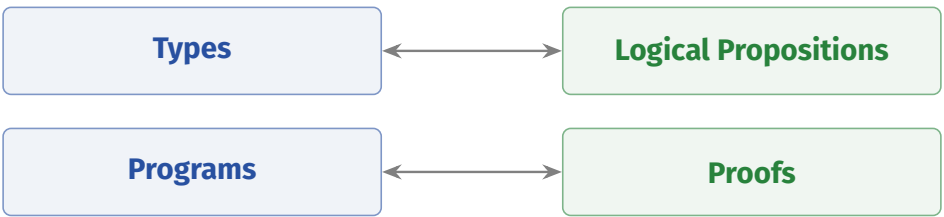


**Programming Languages**

# Curry–Howard Isomorphism

**Syntactic** analogy between:

- Formal systems of logic
- Pure type systems



# Mechanized Proofs and Proof Assistants

- An environment for the interactive development of **formal mathematical proofs** where every step is **mechanically checked** for validity
- **Automatic validity checking** of complex proofs!
- Many available **tools**: Rocq, Agda, Lean, F*, Isabelle/HOL, ACL2, MetaMath, …
- Different **expressive power** of the logical system (e.g. first-order logic vs. higher-order logic)
- Different degrees of proof **automation**
- **AlphaProof**: AI model by DeepMind that writes proofs in Lean (silver medal, IMO 2024)

> The *verification* of a proof in a **formal system of logic** is a process that can be performed **mechanically** and **efficiently**.

# Rocq

- Based on the **Calculus of Constructions** (CoC)
  Thierry Coquand, first release 1989
- **Lambda calculus** with (dependent) **types**
- Based on the **Curry–Howard** isomorphism
- A tool for **formalizing mathematics**
  - Four color theorem, Feit–Thompson theorem, and many more (madiot.fr/coq100)
- A tool for **modeling programming languages**
- A software development environment with **correctness proofs**
  - **CompCert**: fully verified C compiler, **CertiKOS**: verified OS kernels, …



# Course Overview

- **Formal semantics** of functional and imperative languages
- **Type systems** of functional languages
- **Curry–Howard** isomorphism
- Semantic correctness of programs: **formal specifications** and **formal proofs**
- Programming with **dependent types**
- Program **transformations**
- Program **analysis** (symbolic execution, software testing)
- …

# Course Organization

| | |
|---|---|
| **Course site** | Helios |
| **Lectures** | **Friday 11:45 – 14:30** |
| **Assignments** | **Weekly** ($\sim$11, tentative), **10-day** deadline |
| **Exam** | **No** final exam |
| **Office hours** | Wednesday 13:30–15:30, room 1.1.21 or Softlab (1.1.25) |